Reproducing GlareShell: Graph Learning-based PHP Webshell Detection, Revisited under Leakage Control

Dang Minh Tu, Vo Quang Vu, Le Tri Duc
University of Information Technology, Ho Chi Minh City, Vietnam
Vietnam National University, Ho Chi Minh City, Vietnam
{24520033,24522045,24520009}@gm.uit.edu.vn

***Abstract*—GlareShell (Feng et al., Computer Networks 2024) detects PHP webshells by turning each script into an interprocedural control-flow graph of statements, embedding the statements with word-embedding models, adding a risk level per statement and classifying the graph with a GNN; it reports F1 = 0.9887. Neither code nor data were released. We re-implement the pipeline on the same static-analysis tool (with four fixes needed for modern PHP), rebuild the corpus from the paper's sources with provenance, hashes and near-duplicate clusters, and run every experiment with five seeds. On the paper's protocol we reproduce the headline result (F1 0.98 for GCN, GIN and GAT; FastText as best embedding) but not the finer claims: GAT is not better than GCN or GIN, the dimension curve is flat, and the ablation of normalisation and risk weights is within noise. Beyond the paper we find that 77% of test webshells in a random split have a near-duplicate in training, that a TF-IDF + logistic-regression baseline beats every GNN (F1 0.997), that twelve graph statistics without code semantics already reach F1 0.92, that the false-positive rate quadruples on unseen benign projects (1.7%), and that the final model flags 10 of 12 hand-written legitimate standalone scripts as webshells. High scores on this benchmark mostly measure the difference between large framework code and webshell collections, not malicious behaviour.**

***Index Terms*—webshell detection, graph neural networks, PHP, reproducibility, data leakage, dataset quality**

I. Introduction

Webshells are the standard persistence mechanism after a web-application compromise. GlareShell [1] proposes to detect PHP webshells statically by turning each script into an interprocedural control-flow graph (ICFG) of PHP statements, attributing each node with a word embedding of the normalised statement and a risk level, and classifying the graph with a GNN. It reports an F1 of 0.9887 with GAT, above sequence- and tree-based approaches.

This report reproduces GlareShell from its description – neither code nor data were released – and then asks whether the reported performance measures what it claims to measure. Our contributions:

1) **A working re-implementation** on top of the same static-analysis tool (LChecker), including four extractor problems that silently corrupt the graphs if left unfixed (Section III).
2) **A rebuilt, documented corpus** (GlareShell-R) from the paper's sources with provenance, two deduplication levels, near-duplicate clusters, a data card and a label guide (Section IV).
3) **A faithful replication** of the paper's Tables 2 and 4 and Figures 6–8 with five seeds and confidence intervals (Section VI-A).
4) **A leakage and generalisation analysis**: a cluster-disjoint split, a cross-source test on four unseen benign projects, simple baselines, and an error taxonomy (Sections VI-B and VI-C).

Course project for NT230 (Malware Analysis); instructor: Phan The Duy.

a) Summary of findings:

- **RQ1:** the headline performance is reproduced: F1 0.98 on the paper's protocol (paper: 0.97–0.99).
- **RQ2:** the component claims are not: GAT ≈ GCN ≈ GIN, embedding dimension and context window barely matter, and normalisation/risk weights do not significantly help (and hurt on a clean split).
- **RQ3:** 77% of test webshells leak via near-duplicates (F1 −2 points without them); a token TF-IDF + LR baseline is better than every GNN, and graph statistics alone reach F1 0.92.
- **RQ4:** on four unseen benign projects the FPR rises from 0.4% to 1.7%; recall on webshells from new collections is 0.54–0.64; 10 of 12 small legitimate scripts are flagged. The detector has learned "not framework code" rather than "malicious".

II. The malware object: PHP webshells

A. What a webshell is and how it operates

A *webshell* is a server-side script planted in the document root of a web server that gives a remote party interactive control of that server through ordinary HTTP requests (MITRE ATT&CK T1505.003 [2]). It is a *post-exploitation persistence* artefact: it does not compromise the server by itself but is dropped after an initial-access step (unrestricted file upload, LFI/RFI, SQL injection with `INTO OUTFILE`, a vulnerable plugin, stolen FTP/CMS credentials). Once present, the PHP interpreter executes it on every request to its URL, with the privileges of the web-server user. GlareShell [1] places this in an Industrial-Internet setting: the web server is the management and monitoring interface of industrial control systems, so a webshell is a pivot from the IT web tier towards the OT network.

Figure 1 summarises the concepts we use throughout the report. Webshells fall into a few *families* that differ in size and in the evidence they leave in source code:

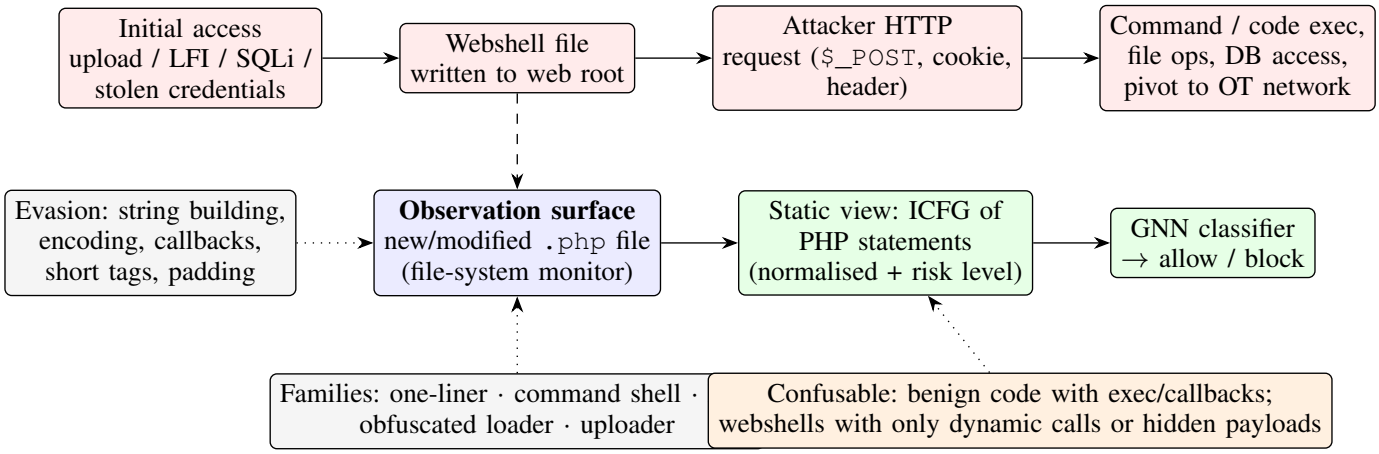


Fig. 1. Concept map: attack chain (red), what the detector observes (blue), the detector (green) and the sources of confusion we analyse (orange).

- **One-line backdoors** ("China Chopper"-style): `<?php @eval($_POST['x']);?>`. One statement; the attacker's client sends the PHP payload in every request.
- **Command shells**: pass request input to `system`/`exec`/`passthru`/`popen`/`` `backticks` ``.
- **Management kits** (c99, r57, WSO, b374k): thousands of lines with file manager, upload, SQL client, port binding, mail bomber, self-removal. Most of their code is ordinary UI code.
- **Obfuscated loaders**: hide the payload in an encoded string (`eval(gzinflate(base64_decode('...')))`) or build the dangerous function name at run time (`$f='as'.'sert'; $f($_REQUEST[k]);`), or route it through callbacks (`array_map($_GET['f'], [$_GET['a']])`, `preg_replace('/.*/e', ...)`).
- **Uploaders / file managers**: no command execution at all – only `move_uploaded_file`, `fopen`/`fwrite`; they are used to stage the next payload.

Variants inside a family usually differ only by password, banner, language of the UI or layout, which is why public collections are full of near-duplicates (Section IV).

B. Threat model

- **Assets**: the web server of an Industrial-Internet management plane and the networks it can reach.
- **Attacker**: remote, already able to write a file into the web root; knows that a static detector may be deployed and can obfuscate, but (in the paper's setting) does not adapt to the specific model.
- **Defender's observation**: the PHP source file at creation/-modification time. No execution traces, no HTTP logs, no file-system context (path, owner, neighbouring files).
- **Decision**: binary, per file, before the file is executed; a false positive blocks a legitimate deployment, a false negative leaves a backdoor.
- **Out of scope**: memory-resident shells, non-PHP shells, adaptive adversarial attacks against the model, legitimately deployed admin tools.

C. Research questions

RQ1 (reproduction) Can GlareShell's reported detection performance (Tables 2 and 4 of [1]) be reproduced from the paper's description, using the paper's data sources and protocol?
RQ2 (components) Do statement normalisation, risk weights, the choice of embedding and the choice of GNN contribute as claimed (Fig. 6–8 of [1])?
RQ3 (validity) How much of the performance is explained by near-duplicate leakage between train and test and by the benign/webshell source confound?
RQ4 (generalisation) How does the detector behave on benign projects and webshell collections that were not part of the training sources, i.e. the false-positive rate a deployment would see?

D. Labels and the features the detector sees

A file is *webshell* if it comes from a public webshell collection and *benign* if it comes from a pinned release of a widely deployed project; labels are source-level (full guide in `docs/LABEL_GUIDE.md`). Table I lists what GlareShell actually observes for each file.

a) Confusable cases: The feature view predicts where the detector should fail, and we test these predictions in Section VI-C: (i) benign framework code is full of *obfuscated*-level calls (`call_user_func_array` for dispatch, `usort` with closures, `preg_replace_callback`) and installers/updaters call `exec`; (ii) webshells that never call a dangerous built-in by name (dynamic calls `$f(...)`, uploaders) get no risk signal; (iii) payloads inside encoded strings are a single opaque statement; (iv) the static ICFG is incomplete for reflection, variable functions and `include` of runtime paths.

III. GlareShell and our re-implementation

A. The original pipeline

GlareShell [1] has three components. **(1) ICFG extractor**: LChecker [3] (built on PHP-Parser [4]) builds one CFG per function/method; Algorithm 1 of the paper connects them through call relations into an interprocedural CFG whose

TABLE I
FEATURE VIEW OF GLARESHELL (PER NODE OF THE ICFG UNLESS STATED).

Feature	Content	Malicious signal it can carry
Statement text	One PHP statement / branch condition, pretty-printed	API used, request input, payload strings
Normalisation	local variables → `$local`, user functions/methods → `user_define`, numbers → `im`	removes names/passwords, keeps built-ins and strings
Embedding	100-d vector of the normalised statement (W2V / FastText / GloVe / D2V)	co-occurrence of risky statements
Risk level	one-hot: 2 = *critical* (`system`, `exec`, `shell_exec`, `passthru`, `popen`, `proc_open`, `pcntl_exec`, backticks); 1 = *obfuscated* (`eval`, `assert`, `preg_replace`, `create_function`, `call_user_func*`, array/sort callbacks, ...); 0 = normal	explicit dangerous capability
Edges	intra-procedural control flow + call/return edges between functions (ICFG)	how input reaches the sink
Graph	mean of final node states (Eq. 6 of [1])	whole-file behaviour

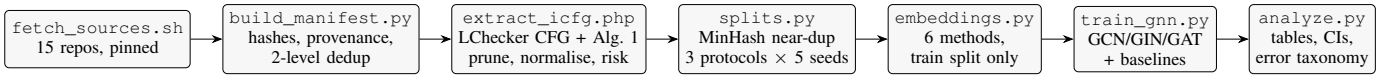


Fig. 2. Reproduction pipeline. Samples are only parsed; nothing is executed.

nodes are PHP statements; "meaningless" statements (constant `echo`, variable initialisation) are pruned. **(2) Node attribution generator**: statements are normalised (user functions → `user-define`, local variables → `local`, numbers → `im`) and embedded with one of six word-embedding methods (Word2Vec CBOW/Skip-gram [5], FastText [6], GloVe [7], Doc2Vec PV-DM/PV-DBOW [8]); a three-level risk weight is concatenated. **(3) GNN classifier**: GCN [9], GAT [10] or GIN [11], mean pooling and an MLP. The paper reports FastText as best embedding (F1 0.9454 with a small GCN) and GAT as best GNN (F1 0.9887) on 3,104 webshells / 10,375 benign files split randomly 8:1:1. Neither code nor data is published.

B. Our implementation

Figure 2 shows the reproduction pipeline; every box is a script in `src/`.

a) ICFG extraction: We tried LChecker first, as the paper did. Its *analysis* (`Main.php`) is a taint analysis for loose comparisons and does not output graphs, but its CFG builder (`ConstructCFG.php`) is reusable as a library. We therefore call LChecker's `ConstructCFG`, `ConstructFunctionCFG` and `ConstructClassCFG` unchanged and implement the rest of the paper on top (`src/php/extract_icfg.php`, 400 lines):

1) **Algorithm 1**: lower the main-script CFG into a node/edge list; for every statement, resolve calls to functions, methods (by name, ≤3 candidates), constructors and closures defined in the same file, lower the callee CFG once and add a call edge (site → callee entry) and a return edge (callee exit → site). Dynamic calls (`$f()`) cannot be resolved statically – as in the paper.
2) **Pruning**: constant `echo`/inline HTML and assignments of constants to variables are removed; their predecessors are linked to their successors so the graph stays connected.
3) **Normalisation**: applied on a copy of the AST (not on text), so string literals keep their content; a function is "user-defined" if it is not a built-in of PHP 8.3 or of a common extension (prefix list).
4) **Risk level**: computed on the original AST from the called built-ins (Table I).

LChecker is a research prototype written for PHP 7 and PHP-Parser 4.3. Applying it to this corpus revealed four problems that the paper does not mention and that we had to fix to obtain meaningful graphs:

- it *silently skips* `namespace`, `try/catch` and `declare` blocks and functions declared inside other statements. Every PSR-style framework file is wrapped in a namespace, so vanilla LChecker yields empty graphs for most of Laravel/-Drupal – a perfect label leak. We flatten these blocks before CFG construction;
- a `for` loop's condition is stored as a PHP array, which crashes pretty-printing (854 files); we keep its last expression;
- PHP 8.3 ships with `short_open_tag=Off`, so files written with bare `<?` tags parse as HTML. This affected 208 webshells and *zero* benign files; we parse with `short_open_tag=On`;
- 2 files crash the PHP interpreter (segfault on deeply nested expressions); they are kept as empty graphs.

We cap an ICFG at 2,000 statements to bound memory (Table III).

b) Embeddings and GNNs: Gensim 4.3.3 provides Word2Vec, FastText and Doc2Vec with default settings (dimension 100, window 5, 5 epochs, `min_count`=1), except FastText's hash-bucket count (200k instead of 2M, for RAM) and PV-DBOW's `dbow_words`=1 (otherwise the statement

vectors are never trained). GloVe is our own PyTorch implementation (x_{max}=100, α=0.75, 25 AdaGrad epochs) because `glove-python` no longer installs. Each file's statements in emission order form one "sentence". **Embeddings are trained only on the training files of each split and seed** (the paper does not say; training them on all files would leak test statements). Unknown statements get a zero vector, except for FastText which composes them from character n-grams. GNNs are implemented with PyTorch Geometric 2.6.1 following Eq. (3)–(6) of the paper; edges are made undirected; the node attribute is [embedding $\|$ one-hot risk]. Table XII (appendix) lists all hyper-parameters; they follow the paper where it gives values (Table 2 setting: 2-layer GCN, hidden 10, batch 10; Table 3 optimum: 3 layers, lr 10^{-3}, batch 256, 120 epochs, hidden 32/8×8 heads/16). Model selection uses the validation F1 (early stopping); the paper does not describe its selection.

IV. Dataset: rebuilding the corpus and measuring its quality

We take the *Dataset Research Track*: besides rebuilding the paper's corpus we release a provenance-documented manifest with two deduplication levels, near-duplicate clusters and two additional evaluation protocols ("GlareShell-R"; data card in `docs/DATA_CARD.md`, schema in `docs/SCHEMA.md`, label guide in `docs/LABEL_GUIDE.md`).

A. Sources and provenance

We cloned every repository of the paper's Table 1 at pinned revisions (Table II). `ysrc/webshell-sample` has been deleted from GitHub; its files survive only inside the Cyc1e183 aggregate [12], a "cleaned" set of 2,917 webshells drawn from 17 collections, which dominates the webshell side. Its files are named by MD5 and carry no origin; matching MD5s against the collections we could clone recovers the origin of only 646 of them (2,271 remain *unknown*). Benign files come from pinned releases (WordPress 6.4.3, Drupal 10.2.5, Laravel skeleton v10.3.3, PHPMailer 6.9.1, doorGets CMS). For the dataset track we add four *held-out* benign projects (Laravel framework v10.48.4, phpMyAdmin 5.2.1, Joomla 5.0.3, Matomo 5.0.3) and two held-out webshell collections (BlackArch, xl7dev). Licenses range from CC0/MIT to GPL; several webshell collections state none, which is one more reason we never redistribute samples publicly (Section VIII).

a) Reproduced composition: After exact-hash deduplication the paper sources yield **3,097 webshells** (paper: 3,104) and 12,930 benign files. The paper reports 10,375 benign files without saying how they were chosen; we subsample proportionally per project (seed 2024) to exactly that size. The webshell count matches the paper within 0.2%, which suggests the paper used *exact-hash* deduplication.

B. ICFG extraction statistics

Table III shows that extraction itself is *label-dependent*: webshells are older, less well-formed and much larger. 139 webshells (4.5%) contain syntax errors that PHP-Parser recovers from, 477 (15.4%) exceed the 2,000-statement cap (benign: 0.4%), and 53 produce an empty graph. These are signals a classifier can exploit without understanding behaviour (graph size), and failure modes that a real deployment must handle (an unparsable upload is itself suspicious). Statement counts of webshells (Figure 3a) span four orders of magnitude, as in the paper's Fig. 5. Extraction takes 11 ms per file on average (P95 40 ms) on one core.

C. Near-duplicates and leakage

Exact hashing is a weak notion of "unique" for scripts that are copied, re-saved with different line endings, re-branded or re-passworded. We measure duplication at two further levels:

- **Whitespace-insensitive hash**: 1,265 of the 2,917 Cyc1e183 files (43%) are byte-identical once whitespace is removed – the "cleaned" dataset was deduplicated on raw bytes only.
- **Near-duplicate clusters**: we represent each file by the *set* of its normalised ICFG statements, generate candidates with MinHash-LSH [13] (128 permutations, 32 bands) and merge pairs whose exact Jaccard similarity is ≥ 0.8 (union-find). The 3,097 paper webshells collapse into **1,368 clusters**; only 394 webshells are singletons and the largest family has 55 members (Figure 3b). Benign files are much less redundant (12,930 files $\rightarrow$ 11,828 clusters). No cluster mixes labels.

Under the paper's protocol (random 8:1:1), **77.0% ± 4.4% of test webshells have a near-duplicate in the training set** (5 seeds; benign: 9.0% ± 1.4%). The paper's test set therefore mostly measures recognition of already-seen families [14], [15].

D. Evaluation protocols

`paper_random` the paper's protocol: exact dedup, 3,097 webshells / 10,376 benign, stratified random 8:1:1.

`clean_group` leakage-controlled: whitespace dedup (1,747 webshells / 10,372 benign); whole near-duplicate clusters are assigned to train, validation or test (8:1:1 by file count, per label). Same sources as the paper.

`cross_source` generalisation: train/validation of `clean_group`; test = held-out projects/collections after removing anything that shares a cluster with the paper sources: 9,091 benign files from four unseen projects and **only 18** unseen webshells. The latter is too small to estimate recall precisely (we give Wilson intervals); `cross_source` is primarily a *false-positive* stress test, which is the quantity that decides deployability.

Each protocol is instantiated with 5 seeds. We could not build a meaningful temporal split: git "first-seen" dates are upload dates, not malware dates – all 2,917 Cyc1e183 files share one commit (2020-11-13, a single zip), 98% of xl7dev and 86% of BlackArch files share one day; only `tennc/webshell` has a spread history (100 distinct days, 2013–2023), and its files are mostly already inside the aggregate. Source-disjointness (`cross_source`) is our substitute.

V. Experimental setup

a) Hardware/software: Intel CPU (16 threads), 7.6 GB RAM, NVIDIA RTX 3050 Laptop 6 GB, WSL2 Ubuntu, PHP 8.3.6, Python 3.11, PyTorch 2.4.1 (CUDA 12.1), PyTorch

TABLE II

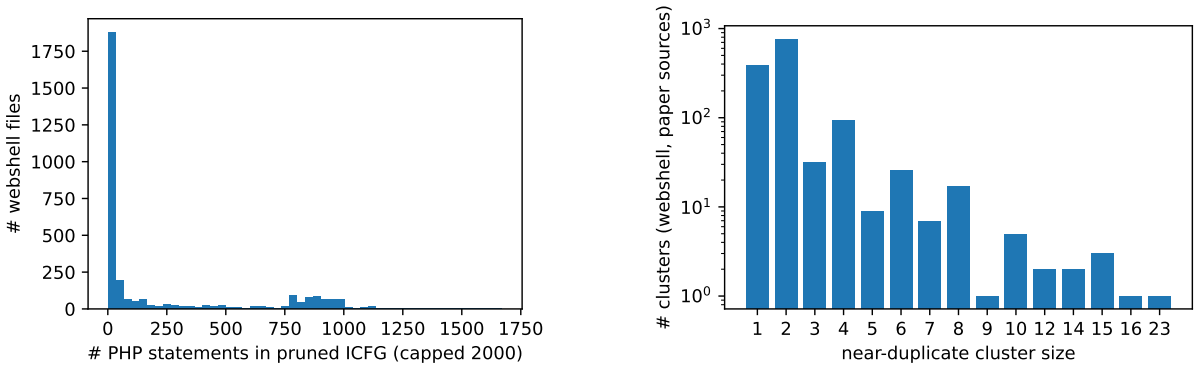
CORPUS FUNNEL PER SOURCE. "EXACT" = FIRST COPY PER SHA-256 (THE PAPER'S DEDUP); "WS" = FIRST COPY PER WHITESPACE-INSENSITIVE HASH. DUPLICATES ARE RESOLVED IN FAVOUR OF THE PAPER SOURCES, SO THE SMALL NUMBERS FOR THE INDIVIDUAL WEBSHELL COLLECTIONS MEAN THAT THEIR FILES ARE ALREADY INSIDE CYC1E183.

Label	Subset	Source	PHP-ext files	contain <?	exact dedup	ws dedup
benign	heldout	joomla/joomla-cms	3105	3105	3104	3103
benign	heldout	laravel/framework	2193	2100	2100	2100
benign	heldout	matomo-org/matomo	3085	3071	2946	2946
benign	heldout	phpmyadmin/phpmyadmin	951	951	951	951
benign	paper	PHPMailer/PHPMailer	124	124	123	123
benign	paper	WordPress/WordPress	1452	1452	1448	1448
benign	paper	doorgets/CMS	1573	1568	1449	1443
benign	paper	drupal/drupal	9868	9866	9860	9860
benign	paper	laravel/laravel	51	50	50	50
webshell	heldout	BlackArch/webshells	201	186	15	1
webshell	heldout	xl7dev/WebShell	282	278	31	18
webshell	paper	Cyc1e183/PHP-Webshell-Dataset	2917	2908	2908	1645
webshell	paper	JohnTroony/php-webshells	119	110	27	3
webshell	paper	bartblaze/PHP-backdoors	184	181	33	10
webshell	paper	tennc/webshell	381	378	129	89

TABLE III

ICFG EXTRACTION OUTCOME (AFTER THE FOUR EXTRACTOR FIXES OF SECTION III).

Label	Subset	Files	Crash	Parse err.	Empty ICFG	Truncated	Median nodes	P90 nodes	ms/file
benign	heldout	9101	0	1	123	7	18	103	6.9
benign	paper	12930	0	1	590	49	14	99	7.9
webshell	heldout	46	0	7	3	4	25	755	26.6
webshell	paper	3097	4	139	53	477	11	875	36.6



(a) Statements per webshell ICFG (cf. paper Fig. 5).

(b) Near-duplicate cluster sizes, paper webshells.

Fig. 3. Corpus structure.

Geometric 2.6.1, Gensim 4.3.3 (paper: Xeon E5-2620 for extraction, i7-12700 + RTX 3060 for training).

b) Metrics: Positive class = webshell. We report the paper's Precision, Recall, TNR, Accuracy, F1 (Eq. 7–11) plus the false-positive rate FPR = 1 − TNR, which matters most in deployment because benign uploads vastly outnumber webshells. Every configuration is run with **5 seeds** (split, embedding and GNN initialisation all depend on the seed; sweeps use 3 seeds); we report mean ± half-width of the 95% Student-t interval. For `cross_source` we additionally give 95% Wilson intervals on the pooled test counts of seed 0.

c) Experiments:

1) **Table 2 replication**: 6 embeddings × 5 seeds, 2-layer GCN, hidden 10, batch 10 (`paper_random`).
2) **Fig. 8 replication**: ablation *none* (raw statements, no risk), *n* (normalised), *n+w* (normalised + risk), FastText + GCN (`paper_random`) and FastText + GAT (`clean_group`).
3) **Fig. 6/7 replication**: embedding dimension 20–120 and context window 2–7 (FastText + GCN, 3 seeds).
4) **Table 4 replication**: GCN/GIN/GAT with the Table 3 optimum, FastText (`paper_random`).
5) **Beyond the paper**: GCN and GAT on `clean_group` and `cross_source`; baselines on all protocols.

d) Baselines: (`src/baselines.py`, same splits): **TF-IDF + LR / XGBoost** on lexical PHP tokens of the raw file

TABLE IV
PAPER TABLE 2 REPLICATION: EMBEDDING METHODS WITH A 2-LAYER GCN (HIDDEN 10, BATCH 10) ON `paper_random`, 5 SEEDS. ΔF1 = OURS − PAPER.

Method	Precision	Recall	TNR	Acc	F1	F1 (paper)	ΔF1
GloVe	0.9268±0.0266	0.9303±0.0320	0.9778±0.0091	0.9669±0.0058	0.9282±0.0130	0.9136	+0.0146
W2V-c	0.9478±0.0356	0.8561±0.0316	0.9857±0.0102	0.9559±0.0108	0.8993±0.0250	0.9373	-0.0380
W2V-s	0.9746±0.0111	0.8619±0.0199	0.9933±0.0030	0.9631±0.0048	0.9147±0.0116	0.9355	-0.0208
D2V-m	0.9594±0.0134	0.8697±0.0311	0.9890±0.0036	0.9616±0.0082	0.9122±0.0198	0.9309	-0.0187
D2V-w	0.9789±0.0204	0.8948±0.0217	0.9942±0.0056	0.9714±0.0080	0.9349±0.0183	0.9293	+0.0056
FastText	0.9868±0.0082	0.9671±0.0111	0.9961±0.0024	0.9895±0.0042	0.9769±0.0094	0.9454	+0.0315

(1–2-grams, hashed to 2^{18} features; χ^2 top-5k for XGBoost) – the classic text approach of prior work [1]; **rules** derived from GlareShell's own risk mechanism (webshell iff a critical call / any risky call / a risky call and request input); and **graph-statistics + XGBoost**: 12 numbers per ICFG (size, density, #critical, #obfuscated, unique-statement ratio, parse error, truncated, ...) and *no code semantics*. If this probe performs well, part of the task is solvable from artefacts of the corpus rather than from behaviour.

VI. RESULTS

All numbers are mean ± 95% CI half-width over 5 seeds (sweeps: 3 seeds), computed by `src/analyze.py` from the per-run JSON files in `results/runs/`. 115 GNN runs took 5.2 GPU-hours on the RTX 3050.

A. Reproducing the paper (RQ1, RQ2)

a) Embedding methods (paper Table 2): Table IV reproduces the paper's first experiment on `paper_random` with the paper's Table-2 GCN. Absolute F1 values land within ±0.04 of the paper and FastText is the best embedding, as in the paper – by a larger margin (0.977 vs. 0.935 for the runner-up D2V-w). The ordering of the remaining five methods is *not* reproduced: the paper ranks W2V-c second and GloVe last, we find GloVe and D2V-w ahead of the Word2Vec variants. These five methods lie within 0.036 F1 of each other with CI half-widths of 0.012–0.025, and the paper's single-run differences (0.914–0.937) are of the same size as our seed-to-seed spread, so its ranking among them is not supported by evidence. All methods trade recall for precision in our runs (TNR $\geq$ 0.978, recall 0.86–0.97); the missed webshells are analysed in Section VI-C.

b) GNN architectures (paper Table 4): With FastText and the Table-3 hyper-parameters (Table V) all three GNNs reach F1 $\approx$ 0.98, again close to the paper (0.968–0.989). The paper's claim that *GAT is the best architecture* is not reproduced: GCN (0.981 ± 0.008), GIN (0.983 ± 0.002) and GAT (0.980 ± 0.012) are statistically indistinguishable, and GAT has the largest seed variance. The Table-3 setting (3 layers, batch 256, up to 120 epochs) improves on the Table-2 setting only marginally for GCN (0.981 vs. 0.977).

c) Embedding dimension and context window (paper Fig. 6–7): The paper reports that F1 first rises with the embedding dimension and then falls (overfitting), and that context size 5 is optimal. In our sweep (Figure 4) F1 is flat: between 0.978 and 0.981 for dimensions 20–120 and between 0.975 and 0.981 for windows 2–7, with CIs of ±0.01–0.02. Window 5 is the nominal maximum, consistent with the paper, but no setting is significantly better than another; a 20-dimensional embedding is as good as a 100-dimensional one. We therefore do *not* reproduce the rise-then-fall shape of Fig. 6.

d) Normalisation and risk weights (paper Fig. 8): Table VI and Figure 5 repeat the paper's ablation. On `paper_random` the order matches the paper (none 0.973 < n 0.977 < n+w 0.977 F1, FPR 0.75% → 0.56% → 0.39%) but all differences lie inside the confidence intervals, so the claim that "both mechanisms improve all metrics" is not statistically supported; recall actually drops slightly with the risk weights. On the leakage-controlled `clean_group` split with GAT the effect reverses: raw statements without normalisation or risk weights are best (0.973 ± 0.009) and normalisation alone is worst (0.955 ± 0.008). Our reading: normalisation removes variable names and literals, which in a random split help to recognise near-duplicates of training webshells; once near-duplicates are removed from the test set, the GNN has less to memorise, and the risk one-hot cannot compensate.

B. Beyond the paper: leakage, baselines and generalisation (RQ3, RQ4)

a) Near-duplicate leakage: In the paper's random 8:1:1 split, 77% (±4%) of the test webshells have a near-duplicate (same whitespace-normalised or MinHash cluster, Section IV-C) in the training set; for benign files the share is 9%. Moving to the cluster-disjoint `clean_group` split lowers F1 by about two points for both GNNs (Table VII: GCN 0.981 → 0.959, GAT 0.980 → 0.963) while the FPR stays at 0.4–0.5%. Part of the paper's score is therefore recognition of variants of known webshells, but the drop is moderate: the detector also generalises to webshell clusters it has not seen, *as long as the benign side comes from the same five projects*.

b) Baselines: A bag of lexical tokens with logistic regression beats every GNN on every protocol (Table VII): F1 0.997 on `paper_random` and 0.992 on `clean_group`, FPR 0.13–0.14%, at a fraction of the cost (seconds on CPU, no ICFG). It is also more accurate than the paper's best reported number (GAT, 0.9887). The GNNs' claimed advantage over "sequence-based" approaches is therefore not reproduced against a properly tuned text baseline. Two baselines probe for shortcuts: twelve *graph statistics* with no code semantics at all (size, density, number of risky statements, parse error,

TABLE V
PAPER TABLE 4 REPLICATION: GNN ARCHITECTURES WITH FASTTEXT AND THE TABLE-3 HYPER-PARAMETERS ON PAPER_RANDOM, 5 SEEDS.

Model	Precision	Recall	TNR	Acc	F1	F1 (paper)	ΔF1
GCN	0.9864±0.0125	0.9755±0.0067	0.9960±0.0037	0.9912±0.0039	0.9809±0.0084	0.9675	+0.0134
GIN	0.9909±0.0077	0.9761±0.0046	0.9973±0.0023	0.9924±0.0010	0.9834±0.0021	0.9767	+0.0067
GAT	0.9833±0.0201	0.9768±0.0044	0.9950±0.0061	0.9908±0.0054	0.9800±0.0117	0.9887	-0.0087

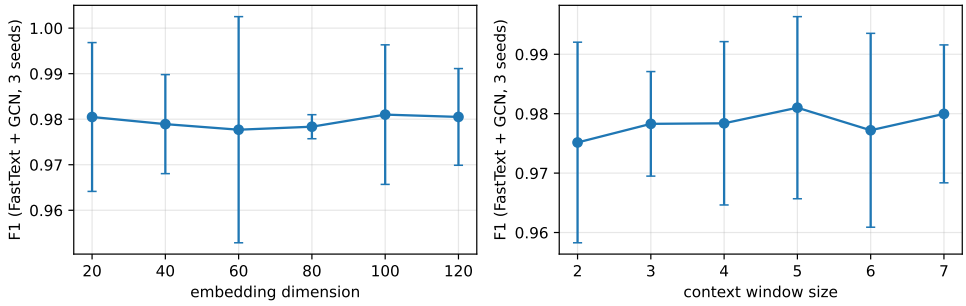


Fig. 4. Embedding dimension (left) and FastText context window (right) vs. F1, FastText + GCN on paper_random, 3 seeds (cf. paper Fig. 6 and 7). Error bars: 95% CI.

TABLE VI
ABLATION OF NORMALISATION (N) AND RISK WEIGHTS (W), FASTTEXT, 5 SEEDS (CF. PAPER FIG. 8). PAPER_RANDOM: TABLE-2 GCN; CLEAN_GROUP: TABLE-3 GAT.

Protocol	Variant	Precision	Recall	TNR	Acc	F1	FPR
paper_random	none	0.9749±0.0166	0.9710±0.0028	0.9925±0.0051	0.9875±0.0035	0.9729±0.0074	0.0075±0.0051
paper_random	n	0.9812±0.0145	0.9723±0.0046	0.9944±0.0043	0.9893±0.0039	0.9767±0.0085	0.0056±0.0043
paper_random	n+w	0.9868±0.0082	0.9671±0.0111	0.9961±0.0024	0.9895±0.0042	0.9769±0.0094	0.0039±0.0024
clean_group	none	0.9836±0.0058	0.9632±0.0193	0.9973±0.0010	0.9924±0.0023	0.9732±0.0085	0.0027±0.0010
clean_group	n	0.9628±0.0158	0.9471±0.0155	0.9938±0.0028	0.9871±0.0024	0.9548±0.0083	0.0062±0.0028
clean_group	n+w	0.9746±0.0241	0.9517±0.0130	0.9958±0.0041	0.9894±0.0023	0.9628±0.0078	0.0042±0.0041

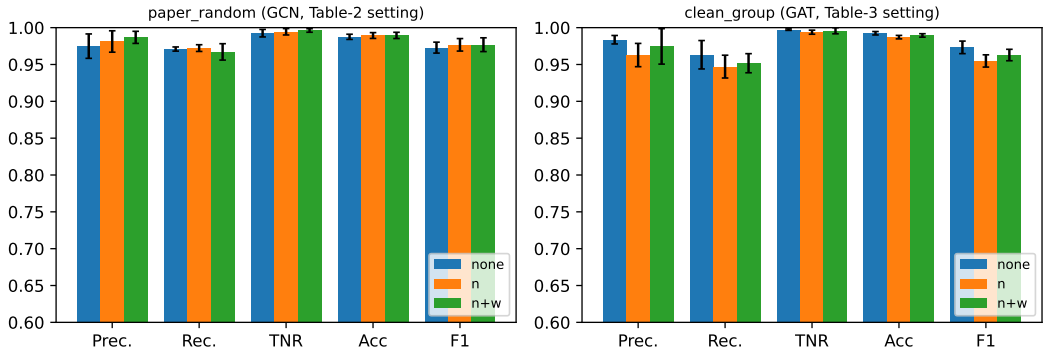


Fig. 5. Ablation of normalisation and risk weights on both protocols (cf. paper Fig. 8).

truncation) reach F1 0.917 on paper_random and 0.825 on clean_group, i.e. most of the separation is available from the shape of the ICFG. Conversely, rules built from GlareShell's own risk lists are weak (F1 0.36–0.64): a webshell is not identified by *which* functions it calls, and many benign framework files call the same functions (10% FPR for "any risky call").

c) Unseen benign projects (deployment FPR): On cross_source the GNNs' FPR rises four-fold, to 1.7% (GAT) and 1.8% (GCN) – about 150 false alarms per 9,091 benign files per run; with seed 0 the 95% Wilson interval is 0.8–1.2% for GAT and 2.1–2.7% for GCN. The rate varies by project (Table VIII): Joomla is worst (3%), Laravel best (0.8%). TF-IDF + LR stays at 0.35%, showing that the jump is not a property of the projects alone but of what the GNN

TABLE VII
GNNs AND BASELINES UNDER THE THREE PROTOCOLS (5 SEEDS). CROSS_SOURCE: TRAINED ON THE FIVE PAPER BENIGN PROJECTS AND CYC1E183, TESTED ON 9,091 FILES OF FOUR UNSEEN BENIGN PROJECTS AND 18 WEBSHELLS FROM COLLECTIONS NOT IN THE PAPER CORPUS.

Model	F1 (paper split)	FPR (paper split)	F1 (clean)	FPR (clean)	FPR held-out benign	Recall novel webshells
GlareShell GCN	0.9809±0.0084	0.0040±0.0037	0.9589±0.0106	0.0054±0.0044	0.0184±0.0072	0.544±0.058
GlareShell GAT	0.9800±0.0117	0.0050±0.0061	0.9628±0.0078	0.0042±0.0041	0.0169±0.0073	0.644±0.125
TF-IDF + LR	0.9974±0.0023	0.0013±0.0011	0.9920±0.0053	0.0014±0.0014	0.0035±0.0002	0.767±0.031
TF-IDF + XGBoost	0.9977±0.0023	0.0012±0.0016	0.9902±0.0082	0.0015±0.0014	0.0134±0.0026	0.922±0.038
Graph-stats + XGBoost	0.9168±0.0076	0.0225±0.0042	0.8245±0.0426	0.0176±0.0041	0.0288±0.0011	0.422±0.079
Rule: critical call	0.4115±0.0391	0.0013±0.0007	0.3594±0.0822	0.0019±0.0015	0.0045±0.0000	0.111±0.000
Rule: any risky call	0.6371±0.0096	0.1040±0.0079	0.5514±0.0769	0.1157±0.0239	0.0998±0.0000	0.222±0.000
Rule: risky call + request input	0.5983±0.0275	0.0135±0.0027	0.5597±0.0826	0.0137±0.0044	0.0099±0.0000	0.111±0.000

TABLE VIII
FALSE-POSITIVE RATE PER UNSEEN BENIGN PROJECT (CROSS_SOURCE, 5 SEEDS).

Held-out project	Files	GlareShell GAT	GlareShell GCN	TF-IDF + LR	Rule: any risky call
joomla/joomla-cms	3103	0.0305±0.0173	0.0317±0.0205	0.0005±0.0004	0.0909±0.0000
laravel/framework	2100	0.0082±0.0032	0.0127±0.0008	0.0025±0.0005	0.0968±0.0000
matomo-org/matomo	2946	0.0099±0.0054	0.0094±0.0025	0.0084±0.0005	0.1122±0.0000
phpmyadmin/phpmyadmin	951	0.0137±0.0067	0.0160±0.0041	0.0011±0.0000	0.0967±0.0000

has learned.

d) Unseen webshells: Only 18 deduplicated webshells come from collections outside the paper corpus, so recall estimates are coarse (seed-0 Wilson interval for 9/18: 29–71%). Even so, recall drops from 0.97 to 0.64 ± 0.13 (GAT) and 0.54 ± 0.06 (GCN). TF-IDF + XGBoost recalls 0.92 of them.

e) Small standalone benign scripts: Every benign training file comes from a large framework or CMS. To test whether the models learned "malicious behaviour" or "not framework code", we wrote 12 ordinary standalone scripts of the kind found on small company web servers (contact form, image upload with MIME check, PDO login, CSV export, JSON to-do API, whitelisted download, admin ping tool with `escapeshellarg`, cache cleanup, guestbook, router, template renderer, a three-line greeting). None of them is malicious. The final demo model (FastText + GAT trained on `clean_group`, F1 0.974 and FPR 0.1% on its own test set) flags **10 of the 12** as webshells, most with $p > 0.98$, including the three-line `hello.php`; TF-IDF + LR trained on the same split flags 6 (Table IX). Both detectors have learned a large part of the population difference – small, procedural, reads `$_GET/$_POST` – rather than malicious behaviour, and the GNN more so than the text baseline.

C. Error analysis

Table X tags every misclassification of GAT (all 5 seeds) with machine-checkable causes; Figure 6 shows seed-0 confusion matrices.

False negatives are dominated by files the pipeline cannot see into. On `paper_random`, 75% of the missed webshells have an *empty ICFG* and 67% a recovered parse error; 86% contain no call from the risk lists. These are PHP 4-era or deliberately malformed files on which PHP-Parser recovers nothing, and encoded droppers whose payload is a string that only becomes code at run time; 23 of the 36 come from Cyc1e183 files of unknown origin. On `clean_group` the share of *tiny* ICFGs (<5 statements) grows (29%): one-line backdoors whose cluster is no longer represented in training. On `cross_source` all 32 misses come from one collection (xl7dev) and 84% read no request input inside the ICFG. Inspecting the 9 misses of seed 0 shows that about half are *label noise*: helper files shipped inside a webshell kit that are not malicious on their own (`phpinfo.php`, the MySQL configuration, database class and zip helper of a database-dump kit); the rest are genuine shells (a mini command shell, a WordPress backdoor, a JPEG-embedded payload, a sniffer). Source-level labels thus understate recall on real shells and penalise detectors that are right.

False positives on the in-distribution splits are rare (26 over 5 seeds on `paper_random`) and concentrated in two projects, doorgets CMS and WordPress, typically small entry-point files that read request input or use callbacks (`call_user_func`, `preg_replace_callback`). On `cross_source` there are 769 (5 seeds), 62% of them in Joomla; 47% carry *no* risk tag at all and 28% are tiny ICFGs. These are not "suspicious" files: they are small files that look different from the five training projects: in Joomla, half of the 254 distinct flagged files are layout templates (`layouts/`, `tmpl/`) that mix HTML and short PHP blocks, and a third are small event or exception class stubs (`libraries/src/Event/...`). Together with the probe above this is the most important finding of this report: the detector's false-positive behaviour is driven by *distance from the benign training projects*, not by risky code.

D. Efficiency

ICFG extraction takes 11 ms per file on average (95th percentile 40 ms) on one CPU core, and scoring a graph takes milliseconds, so the per-upload cost is compatible with the paper's file-system-monitor deployment. Loading the FastText model takes about 45 s once at start-up (`src/detect.py`).

TABLE IX
WEBSHELL PROBABILITY FOR 12 HAND-WRITTEN BENIGN STANDALONE SCRIPTS (`tests/probe_benign/`); MODELS TRAINED ON `clean_group`, SEED 0. BOLD: FLAGGED ($p \geq 0.5$).

Script	GAT	TF-IDF+LR	What it does
hello.php	**1.000**	**0.924**	echo greeting with `htmlspecialchars`
contact_form.php	**0.998**	**0.760**	validated form + `mail()`
download.php	**0.999**	0.373	whitelisted file download
export_csv.php	**0.996**	**0.569**	PDO query → CSV
guestbook.php	**0.998**	**0.916**	append-only text file
image_upload.php	**0.982**	0.252	MIME/size-checked upload
login.php	**0.930**	**0.651**	PDO + `password_verify`
ping_tool.php	**0.992**	**0.628**	`shell_exec` with validated IP + `escapeshellarg`
router.php	**0.994**	0.146	static route table + `include`
template_render.php	**0.999**	0.201	`extract` + `include` template
cleanup_cron.php	0.376	0.490	delete old cache files
json_api.php	0.204	0.377	JSON to-do list API
Flagged	**10/12**	**6/12**	

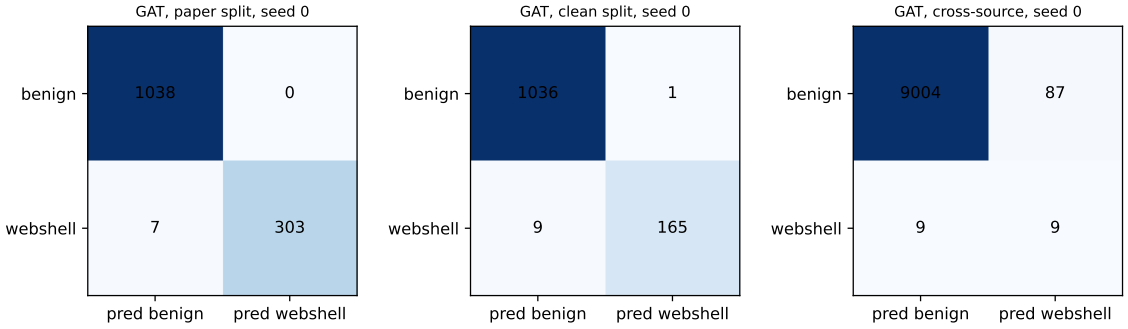


Fig. 6. Confusion matrices of GAT, seed 0, on the three protocols.

TABLE X
ERROR TAXONOMY OF GAT, 5 SEEDS POOLED (TOP-5 TAGS PER ERROR TYPE; TAGS ARE NOT EXCLUSIVE).

Protocol	Error	Tag (non-exclusive)	Count (5 seeds)	Share
paper_random	FN	no risky built-in call	31	86%
paper_random	FN	no request input in ICFG	30	83%
paper_random	FN	empty ICFG	27	75%
paper_random	FN	parse error (recovered)	24	67%
paper_random	FN	tiny ICFG (<5 statements)	3	8%
paper_random	FP	benign reads request input	12	46%
paper_random	FP	no tag	9	35%
paper_random	FP	benign uses callback/eval-like call	6	23%
paper_random	FP	tiny ICFG (<5 statements)	3	12%
paper_random	FP	benign uses critical call	1	4%
clean_group	FN	no request input in ICFG	28	67%
clean_group	FN	no risky built-in call	26	62%
clean_group	FN	parse error (recovered)	20	48%
clean_group	FN	empty ICFG	16	38%
clean_group	FN	tiny ICFG (<5 statements)	12	29%
clean_group	FP	no tag	7	32%
clean_group	FP	benign uses callback/eval-like call	7	32%
clean_group	FP	tiny ICFG (<5 statements)	7	32%
clean_group	FP	benign reads request input	4	18%
clean_group	FP	benign uses critical call	1	5%
cross_source	FN	no request input in ICFG	27	84%
cross_source	FN	no risky built-in call	21	66%
cross_source	FN	parse error (recovered)	10	31%
cross_source	FN	empty ICFG	5	16%
cross_source	FN	no risky built-in call (dynamic call $f())	4	12%
cross_source	FP	no tag	360	47%
cross_source	FP	tiny ICFG (<5 statements)	215	28%
cross_source	FP	benign reads request input	108	14%
cross_source	FP	benign uses callback/eval-like call	93	12%
cross_source	FP	benign uses critical call	23	3%

VII. DISCUSSION

A. What was reproduced, improved, or not reproduced

Table XI separates what we could confirm from what we improved and what we could not reproduce. Colours: **reproduced**, **improved / extended**, **not reproduced**.

B. Threats to validity

a) Internal validity (does our pipeline do what the paper did?):

- *Unreleased implementation.* ICFG construction above LChecker's CFG builder, the pruning rules, the built-in function list, the risk lists and the statement text format are our reading of the paper. Different choices change the vocabulary (we obtain 484k distinct normalised statements) and therefore the embeddings.
- *Extractor fixes.* We changed LChecker's behaviour on namespaces, try/catch, `for` conditions and short tags (Section III). Each fix was needed to avoid graphs that are empty for one class only; if the authors ran vanilla LChecker on PHP 8 their graphs were partly empty, and their numbers are not directly comparable to ours in either direction.
- *Unspecified training details* (epochs for Table 2, early stopping, model selection, embedding training corpus, threshold). We fixed them before looking at test results and report all of them (Table XII).

TABLE XI
VERDICT PER CLAIM OF THE PAPER AND PER ADDITION OF THIS REPORT.

Claim / item	Our evidence	Verdict
ICFG + embedding + GNN detects webshells with F1 $\approx$ 0.95–0.99 (Tables 2, 4)	F1 0.90–0.98 (Table 2 setting), 0.980–0.983 (Table 4 setting) on the paper's protocol, 5 seeds	**Reproduced**
FastText is the best embedding (Table 2)	0.977 $\pm$ 0.009, best of six by $>$0.04	**Reproduced**
Ranking of the other five embeddings	within 0.036 F1, overlapping CIs; order differs	**Not reproduced**
GAT is the best GNN (Table 4)	GCN 0.981, GIN 0.983, GAT 0.980, overlapping CIs	**Not reproduced**
F1 rises then falls with embedding dimension (Fig. 6)	flat 0.978–0.981 for 20–120	**Not reproduced**
Context window 5 is optimal (Fig. 6–7)	nominal maximum at 5, not significant	**Partly reproduced**
Normalisation and risk weights improve all metrics (Fig. 8)	same order on `paper_random` but within CIs; reversed on `clean_group`	**Not reproduced**
GNN outperforms sequence/text approaches	TF-IDF + LR: F1 0.997 (`paper_random`), 0.992 (`clean_group`), lower FPR everywhere	**Not reproduced**
Per-file cost suitable for online monitoring	11 ms extraction + ms scoring per file	**Reproduced**
Working extractor on PHP 8 code	four LChecker fixes, documented	**Improved**
Documented corpus with SHA-256, provenance, near-dup clusters	GlareShell-R, data card, label guide	**Extended**
Leakage-controlled and cross-source evaluation	77% of test webshells leak in the paper's split; F1 -2 points on `clean_group`; FPR $\times 4$ on unseen projects	**Extended**
Shortcut probes	graph statistics alone F1 0.92; 10/12 benign standalone scripts flagged	**Extended**
Standalone detector / directory monitor	`src/detect.py`	**Extended**

- *Resource-driven deviations*: 2,000-statement cap (15% of webshells truncated), FastText 200k buckets, own GloVe implementation.

b) Construct validity (do the labels and metrics measure webshell detection?):

- *Source-level labels.* "Benign" means "file of a large, modern framework/CMS release", "webshell" means "file of a public collection". The two populations also differ in age (PHP 4/5 vs. PHP 8 style), size, structure (procedural vs. namespaced classes) and parseability (Table III). A classifier can reach high scores by separating these *populations* rather than by recognising malicious behaviour; the graph-statistics probe and the hand-written standalone benign scripts (Section VI-B) quantify this.
- *Label noise.* Collections contain non-malicious helper files; benign projects contain test fixtures and installers with webshell-like capabilities. We did not relabel by hand.
- *Class prior.* 23% webshells in test is unrealistically high for a file-upload monitor, so precision and F1 overstate deployment performance; FPR on the 9,091 held-out benign files is the more relevant number.

c) External validity (does it generalise?):

- Only five benign projects in training; four more in testing. Industrial web servers (the paper's motivating setting) run vendor-specific, often small, procedural PHP applications that resemble neither population.
- Only 18 webshells are novel w.r.t. the paper sources; recall on unseen families is therefore estimated with very wide intervals. No temporal split was possible (Section IV-C).
- No adaptive attacker: renaming variables and functions is neutralised by normalisation, but string building, dynamic calls and payload encoding already defeat the risk mechanism for many collected samples.

d) Conclusion validity (are the differences real?): Five seeds give 95% intervals of roughly $\pm$0.01–0.03 F1 for the GNNs; several differences reported in the paper (e.g. between embedding methods, 0.91–0.95 F1 in a single run) are of the same order and cannot be distinguished without repeated runs. We never select hyper-parameters on the test set.

VIII. REPRODUCIBILITY, SAFETY AND ETHICS

a) Handling live malware: The corpus contains working webshells. Our rules (also in `README.md`): samples are *only parsed* by PHP-Parser or read as bytes – no script includes, evaluates or serves them, and the extractor is invoked on lists of paths, never inside a web server; `samples/` and every derived file that contains code fragments (`data/processed/`) are git-ignored; the Docker image runs with `-network none` and the samples mounted read-only; for hand-over to graders, `scripts/pack_samples.sh` builds a password-protected archive (password `infected`, the customary convention) with a SHA-256 checksum, to be shared privately. The development VM can be rolled back from a snapshot after use. The repository publishes only hashes, paths, aggregate statistics and code. We do not release any new webshell, payload or evasion technique; the evasion classes we discuss are already documented in the public collections.

b) Licenses: Benign projects are GPL/LGPL/MIT and are re-fetched from upstream rather than redistributed. Several webshell collections state no license; we reference them by URL and commit only.

c) Reproducibility artefacts: `scripts/fetch_sources.sh` (pinned revisions), `scripts/setup_tools.sh` (LChecker @c8a3646, PHP-Parser v4.19.4, no composer needed), `requirements.txt` (exact versions), `Dockerfile`, `scripts/reproduce.sh` (end-to-end), the manifest with SHA-256 of every sample (integrity check), the splits as JSON, every run's configuration,

metrics and per-file predictions as JSON (`results/runs/`), and `src/analyze.py`, which regenerates every table and figure of this report. Total runtime on our laptop: dominated by the 115 GNN runs (5.2 GPU-hours measured); ICFG extraction costs 11 ms per file, i.e. under 5 CPU-minutes for the whole corpus; baselines and analysis add minutes.

IX. Conclusion

We reproduced GlareShell without the authors' code or data. The pipeline works and, on the paper's own protocol, the reported F1 of about 0.98 is reproducible. The paper's finer conclusions – which embedding is second best, which GNN is best, how dimension and context size matter, and that normalisation and risk weights help – are not supported once seeds and confidence intervals are taken into account.

More important is what the benchmark measures. Benign files come from five large frameworks, webshells from public collections; 77% of test webshells have a near-duplicate in training; a lexical baseline is more accurate than any GNN; and the models flag most small, legitimate standalone scripts. A detector trained this way would raise frequent false alarms on exactly the small vendor applications that run on industrial web servers. For a practical detector we would (i) collect benign code that resembles the deployment (small procedural applications, admin tools, upload handlers), (ii) evaluate on cluster- and source-disjoint splits by default, (iii) relabel helper files inside webshell kits, and (iv) report FPR on unseen projects alongside F1. All code, splits, per-run predictions and the document trail needed to rerun this study are in the repository.

References

[1] P. Feng, D. Wei, Q. Li, Q. Wang, Y. Hu, N. Xi, and J. Ma, "GlareShell: Graph learning-based PHP webshell detection for web server of industrial internet," *Computer Networks*, vol. 245, p. 110406, 2024.

[2] MITRE ATT&CK, "Server software component: Web shell (T1505.003)," https://attack.mitre.org/techniques/T1505/003/, 2024.

[3] P. Li and W. Meng, "LChecker: Detecting loose comparison bugs in PHP," in *Proceedings of The Web Conference (WWW)*, 2021.

[4] N. Popov, "PHP-Parser: A PHP parser written in PHP," https://github.com/nikic/PHP-Parser, v4.19.4, 2024.

[5] T. Mikolov, I. Sutskever, K. Chen, G. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," in *NeurIPS*, 2013.

[6] P. Bojanowski, E. Grave, A. Joulin, and T. Mikolov, "Enriching word vectors with subword information," *TACL*, vol. 5, pp. 135–146, 2017.

[7] J. Pennington, R. Socher, and C. Manning, "GloVe: Global vectors for word representation," in *EMNLP*, 2014.

[8] Q. Le and T. Mikolov, "Distributed representations of sentences and documents," in *ICML*, 2014.

[9] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *ICLR*, 2017.

[10] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *ICLR*, 2018.

[11] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?" in *ICLR*, 2019.

[12] Cyc1e183, "PHP-Webshell-Dataset," https://github.com/Cyc1e183/PHP-Webshell-Dataset, commit 59ff14f, 2021.

[13] A. Z. Broder, "On the resemblance and containment of documents," in *Compression and Complexity of Sequences*, 1997.

[14] M. Allamanis, "The adverse effects of code duplication in machine learning models of code," in *Onward! (SPLASH)*, 2019.

[15] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," in *USENIX Security Symposium*, 2022.

Appendix A
Hyper-parameters

TABLE XII
HYPER-PARAMETERS. "PAPER" = VALUE STATED IN [1]; EVERYTHING ELSE IS OUR CHOICE.

Item	Value	Source
Split	8:1:1 stratified (`paper_random`), cluster-grouped (`clean_group`)	paper / ours
Embedding dim / window	100 / 5 (sweeps 20–120 / 2–7)	paper (default, Fig. 6–7)
Embedding epochs	gensim default (5; Doc2Vec 10), GloVe 25	default
Table 2 GNN	GCN, 2 layers, hidden 10, batch 10, lr 10^{-3}	paper (lr ours)
Table 2 epochs	$\leq$50, early stop on val. F1 (patience 10)	ours
Table 3/4 GNN	3 layers, lr 10^{-3}, batch 256, $\leq$120 epochs (patience 30)	paper (patience ours)
Hidden	GCN 32, GIN 16, GAT 8×8 heads	paper
Optimiser / loss	Adam / cross-entropy, threshold 0.5	ours
Pooling	mean (Eq. 6), then MLP $d \to d \to 2$	paper / ours
ICFG cap	2,000 statements	ours
Near-dup threshold	Jaccard ≥ 0.8 on normalised statement sets	ours
Seeds	0–4 (sweeps 0–2)	ours

Appendix B
Mapping to the course rubric

Criterion	Where
1. Malware object & RQ	Section II: concept map (Figure 1), families, threat model, RQ1–4, labels, feature table, confusable cases
2. Reproduction	Section III, Section VI-A: code in `src/`, environment, comparison tables with the paper, ablation, sweeps, 5 seeds
3. Dataset	Section IV, `docs/DATA_CARD.md`, `LABEL_GUIDE.md`, `SCHEMA.md`, manifest, cluster file, splits, fetch/clean scripts
4. Experiments & critique	Sections VI and VII: P/R/F1/FPR with CIs, confusion matrices, error taxonomy, cross-source evaluation, threats to validity
5. Reproducibility & ethics	Section VIII, `README.md`, `Dockerfile`, checksums, `pack_samples.sh`